

CANDEX HANDBOOK

# Déploiement

Comment Candex est livré — l'API en stack Docker Compose par environnement, CI/CD, le reverse proxy, et les releases de l'app mobile sur les stores.

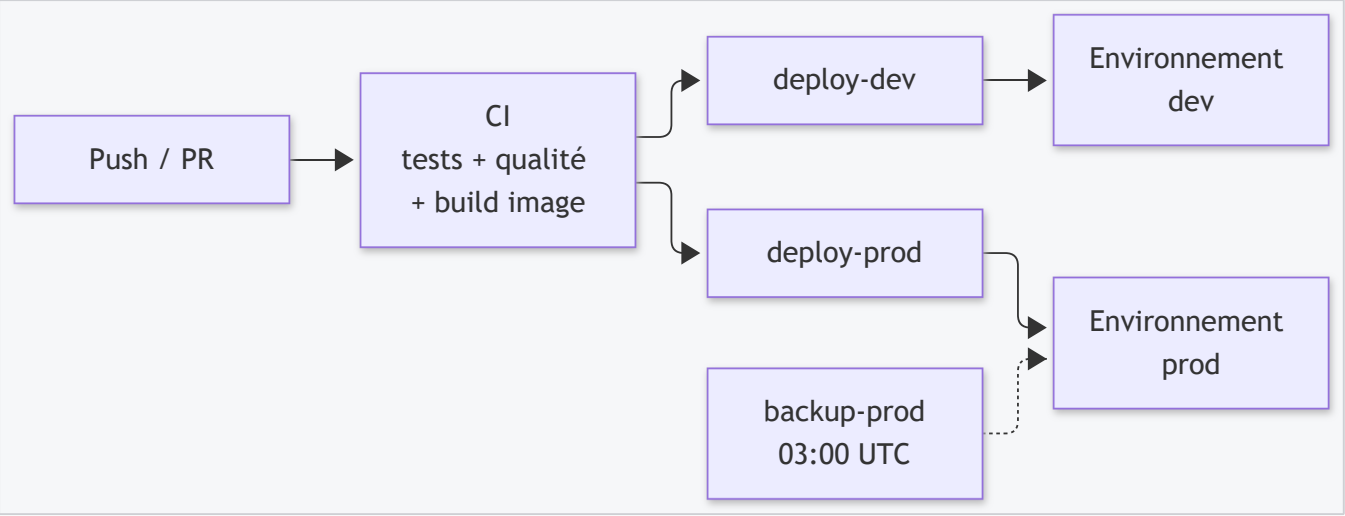
Rédigé par
**Eliott Barbet**

Version 0.23.0
novembre 2026
candex.app

# Déploiement

Voici comment Candex est construit, livré et exploité. L'API tourne en stack **Docker Compose** par environnement ; l'app mobile est livrée sur les stores avec **Fastlane**. Pour la lancer sur votre machine, voir Installation locale ; pour le système autour, Architecture & Vue d'ensemble.

## Le pipeline en un coup d'œil




## Environnements

Chaque environnement est sa propre stack Compose (un conteneur API + un conteneur PostgreSQL 17) :

| Environnement | Fichier compose | Exposition |
|---|---|---|
| local | `compose.local.yaml` | Stack jetable via `scripts/candex.sh` (Installation locale) |
| dev | `compose.dev.yaml` | `api-dev` + `db-dev` ; API sur l'hôte `8081`, debug sur `5005` |
| prod | `compose.prod.yaml` | `api-prod` + `db-prod` ; **aucun port publié** — atteint via le reverse proxy |

Chaque conteneur API tourne avec `SPRING_PROFILES_ACTIVE` réglé sur son environnement, redémarre `unless-stopped`, et garde des logs `json-file` rotés.

# L'image de l'API

L'API est construite par un `Dockerfile` multi-stage — compilée avec Maven, puis exécutée sur un JRE léger :

```
FROM maven:3.9.9-eclipse-temurin-21 AS build
# ... mvn -B -DskipTests package
FROM eclipse-temurin:21-jre
COPY --from=build /app/target/*.jar /app/app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

La configuration est entièrement pilotée par l'environnement (base, sécurité, console SQL, Uploadcare) — voir Installation locale pour les variables. Aucun secret n'est intégré à l'image.

# Exploiter une stack

`scripts/candex.sh` opère une stack dev ou prod en cours d'exécution depuis le serveur — sans l'arrêter :

```
./scripts/candex.sh psql <dev|prod>        # une session psql
./scripts/candex.sh logs <dev|prod>        # suivre les logs de l'API
./scripts/candex.sh dump <dev|prod>        # pg_dump de la base
./scripts/candex.sh exec <dev|prod> "<SQL>"
```

# CI/CD

Des GitHub Actions pilotent l'intégration et la livraison (`.github/workflows`) :

| Workflow | Rôle |
| --- | --- |
| `ci.yml` | Garde-fou qualité sur push / PR — build, tests, lint (et Qodana) |
| `deploy-dev.yml` • `deploy-prod.yml` | Déployer l'API en dev / prod |
| `backup-prod.yml` | Sauvegarde planifiée de la base de production |

Le dashboard a son propre garde-fou `ci.yml`.

# Reverse proxy & TLS

> **À confirmer**
>
> L'API de production ne publie aucun port ; le point d'entrée public et le TLS sont gérés par un **reverse proxy au niveau serveur** (une variable `CADDY_EMAIL` pointe vers Caddy). La configuration exacte du proxy vit hors de ces dépôts et n'est pas encore documentée ici.

# Releases de l'app mobile

L'app est livrée sur le **Play Store** et l'**App Store** avec **Fastlane**, déclenchée manuellement depuis GitHub Actions. Le bump de version et le changelog restent locaux et relus par un humain — la CI ne déploie qu'un tag existant :

```
npm run release 1.2.0            # bump + tag annoté x.y.z (message = changelog)
git push && git push origin 1.2.0
# GitHub ▸ Actions ▸ « Deploy to Stores » ▸ Run workflow (store : both / play / appstore)
```

`deploy.yml` (un `workflow_dispatch` manuel) construit le tag choisi et le soumet via les lanes Fastlane `android deploy` (AAB → Play production) et `ios deploy` (IPA → App Store

Connect).

# Où aller ensuite

- Sauvegardes de la base → Sauvegardes
- La lancer sur votre machine → Installation locale
- L'API elle-même → Back-end